ARCHITECTURE LOGICIELLE · SÉANCE 5

TP 5 · Event Storming QueenBee
Du cahier des charges aux bounded contexts

Faire émerger les événements, le langage, les frontières et la context map de la plateforme apicole de BeeAfrica, puis les prolonger dans votre spécialité (Dev, Data ou Cyber)

Classe	Master 1 · Tronc Commun (EASRSI · EADL) · INGETIS
Date	Vendredi 23 octobre 2026 · TP de 11h15 à 12h20, bilan de 12h20 à 12h30
Travail	Un mur par groupe de 2 à 3 binômes **de la même spécialité** ; livrables déposés par chaque binôme
Support	Cahier des charges QueenBee (chapitres 3, 4, 6 et 7) · post-it de 7 couleurs ou tableau Miro
Rendu	À 12h20 : photo du mur, `glossaire.md`, `context-map.md` · le 30/10/2026 : le livrable de spécialité, dans le dossier

---

Sommaire

1. Ce qui vous est demandé : objectifs, livrables, déroulé, organisation
2. Partie A · Préparation (11h15 – 11h20)
3. Partie B · Event Storming en quatre temps (11h20 – 11h48)
4. Partie C · Frontières et glossaire (11h48 – 11h56)
5. Partie D · Context map (11h56 – 12h05)
6. Partie E · Suite par spécialité : Dev, Data, Cyber (12h05 – 12h20, puis dossier)
7. Rendu, auto-vérification et grille d’évaluation
8. Annexes : cas d’utilisation, événements de départ, modèles de fichiers, aide-mémoire Mermaid

1 · Ce qui vous est demandé

Le contexte

QueenBee est la plateforme de gestion apicole de l'association BeeAfrica : 3 pays, 8 sites ruchers, 400 ruches connectées, 25 coopératives. Des capteurs mesurent le poids, la température, l'humidité et l'activité acoustique ; une passerelle LoRaWAN par site transmet les mesures, souvent en différé ; un modèle prédit les risques (essaimage, affaiblissement) ; les apicultrices et les coordinateurs agissent sur le terrain ; le gestionnaire rend compte aux bailleurs.

Dans ce module, **on ne développe pas QueenBee** : on en conçoit l'architecture. Le TP 5 en est la première étape. Ce que vous produisez aujourd'hui sert directement la séance 6 (modèle C4 et ADR) et le dossier du 30/10.

Les objectifs

1. Explorer tout le domaine QueenBee à partir des cas d'utilisation, sous forme d'**événements métier** ordonnés dans le temps.
2. Fixer le **langage omniprésent** : un glossaire où chaque mot ambigu est tranché.
3. Découper le domaine en **bounded contexts**, en traçant les frontières là où le vocabulaire change de sens.
4. Dessiner la **context map** : qui publie quoi, qui dépend de qui, et avec quel type de relation.
5. Prolonger le travail dans votre **spécialité** : un agrégat (Dev), un contrat de données (Data) ou des zones de confiance (Cyber).

Les livrables

Livrable	Contenu attendu	Quand
Le mur (photo ou export)	Au moins 40 événements, ordonnés, avec commandes, acteurs, systèmes, politiques, agrégats et points chauds ; les frontières des contextes tracées	12h20
`docs/glossaire.md`	Au moins 15 termes : définition, contexte, nom dans le code ; les mots à plusieurs sens ont une ligne par sens	12h20
`docs/bounded-contexts.md`	5 à 8 contextes : responsabilité, type de sous-domaine, agrégats, événements publiés et consommés, UC sources	12h20 (peut être complété pour le dossier)
`docs/context-map.md`	La context map en Mermaid, chaque flèche typée	12h20
`docs/points-chauds.md`	Les questions ouvertes du mur ; celles qui demandent une décision deviendront des ADR en séance 6	12h20
Livrable de spécialité	Dev : agrégat Alerte codé et testé · Data : contrat « Mesure validée » · Cyber : zones de confiance	Amorcé à 12h20, terminé le 30/10

Hors périmètre. Coder la plateforme, produire le MCD complet, les pipelines CI/CD ou les modèles de machine learning : cela relève du projet M1 sur quatre mois. Ici, on prépare son architecture.

Le déroulé minuté

11h15 A · Préparation	**11h20** B1 · Chaos	**11h27** B2 · Chronologie	**11h34** B3 · Causes	**11h42** B4 · Agrégats	**11h48** C · Frontières et glossaire	**11h56** D · Context map	**12h05** E · Spécialité

Les points de passage sont annoncés à voix haute : 11h34 (chronologie terminée), 11h48 (agrégats posés), 11h56 (glossaire), 12h05 (context map). Si votre groupe est en retard, passez quand même à l'étape suivante : un mur incomplet mais découpé vaut mieux qu'un mur parfait sans frontières. **Bilan à 12h20 :** chaque groupe présente en une minute un mot qui a changé de sens sur son mur et la frontière qu'il en a tirée.

1 · Ce qui vous est demandé (suite) : organisation

Groupes et rôles

Chaque groupe réunit 2 à 3 binômes **de la même spécialité** autour d'un seul mur. Trois rôles tournent à chaque étape :

Rôle	Responsabilité
Facilitateur	Lit la consigne de l'étape, fait parler tout le monde, tranche quand le groupe tourne en rond (en posant un point chaud).
Gardien du temps	Annonce la mi-temps et les deux dernières minutes de chaque étape ; refuse les débats de plus de 2 minutes.
Scribe du glossaire	Note en direct, dans `glossaire.md`, chaque mot discuté et le sens retenu ; note les points chauds dans `points-chauds.md`.

Matériel : la légende des sept couleurs

Événement — fait métier au passé : « Récolte déclarée »

Commande — intention, à l'impératif : « Déclarer une récolte »

Acteur — qui lance la commande

Agrégat — objet qui reçoit la commande et garantit les règles

Politique — « quand <événement>, alors »

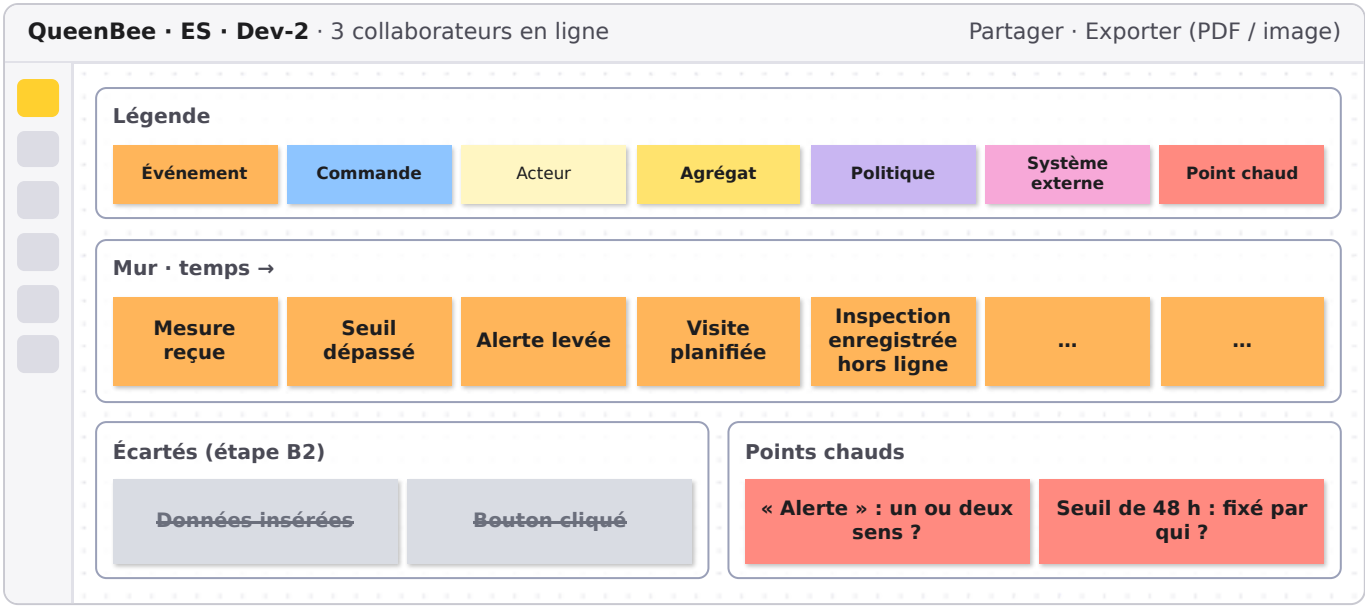
Système externe — système extérieur : capteur, modèle, météo

Point chaud — question ouverte, désaccord, risque

~~Écarté~~ — post-it retiré à l'étape 2

Exemple d'interface : le tableau Miro préparé

Si vous travaillez en ligne, préparez le tableau comme ci-dessous avant 11h20 : un cadre « Légende », un cadre « Mur » avec une flèche du temps, un cadre « Écartés » et un cadre « Points chauds ». Nommez le tableau `QueenBee · ES · <spécialité>-<n° de groupe>`.



Sans Miro : un mur ou une grande table, des post-it de couleur (ou des feuilles A6 avec un code couleur au feutre), et une bande de ruban adhésif pour la flèche du temps. Photographiez le mur à chaque fin d’étape.

Partie A · Préparation 11h15 – 11h20 · 5 min

11h15	11h20	11h27	11h34	11h42	11h48	11h56	12h05
A · Préparation	B1 · Chaos	B2 · Chronologie	B3 · Causes	B4 · Agrégats	C · Frontières et glossaire	D · Context map	E · Spécialité

Ce qu'il faut faire

1. Former le groupe (2 à 3 binômes de la même spécialité), choisir le premier facilitateur, le gardien du temps et le scribe.
2. Créer le dépôt ou le dossier de rendu de chaque binôme avec l'arborescence de la section 7 (fichiers vides).
3. Préparer le mur : légende, flèche du temps, zone « Écartés », zone « Points chauds » (voir l'exemple Miro).
4. Se répartir la relecture des cas d'utilisation (annexe A1) : chaque personne prend 2 ou 3 UC et y **surligne les faits qui se produisent**.

Comment repérer un événement dans un cas d'utilisation

Cherchez les verbes qui décrivent un changement dans le monde, et mettez-les au passé composé. Exemple sur UC-A03 (déclaration d'une récolte) :

Phrase du cahier des charges	Événement candidat
« Il saisit le volume de miel récolté et la date »	Récolte déclarée
« Le système compare la déclaration à la prédiction de rendement »	Écart de rendement calculé
« La coopérative et le coordinateur sont notifiés »	Coordinateur notifié · Coopérative notifiée

Attendu à 11h20 : chaque personne arrive à l'étape B1 avec 8 à 12 événements candidats repérés dans ses UC.

Partie B · Event Storming en quatre temps 11h20 – 11h48 · 28 min

B1 · Le chaos 11h20 – 11h27 · 7 min

Objectif : sortir le plus de faits possible, vite, sans se censurer.

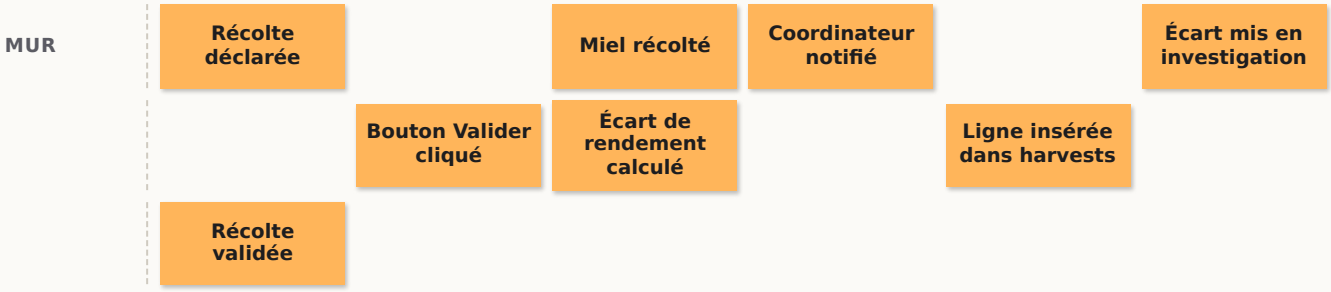
1. Chacun écrit en silence des post-it **orange** : un fait métier au passé par post-it, en 2 à 5 mots.
2. On colle les post-it n'importe où sur le mur, sans ordre et sans discussion.
3. Partir des UC relus en A, puis des événements de départ (annexe A2), puis de son expérience : « que peut-il arriver d'autre à une ruche, une passerelle, une récolte, un rapport ? »

Résultat attendu : au moins 50 post-it orange pour le groupe, doublons compris.

Bons et mauvais post-it

Mauvais post-it	Pourquoi	Mieux
Insérer la mesure en base	Action technique, à l'impératif	Mesure enregistrée
Alerte	Un nom, pas un fait	Alerte levée · Alerte résolue
L'apicultrice clique sur Valider	Détail d'interface	Inspection enregistrée
Envoyer un SMS	C'est une commande (bleu), pas un événement	Apicultrice notifiée
Erreur 500 sur l'API	Technique, invisible pour le métier	Synchronisation échouée
Données reçues	Trop vague	Mesure reçue · Trame rejetée

Exemple d'interface : le mur à la fin du chaos (tranche « Récolte », vue en cours)



Trois post-it sont à écarter à l'étape suivante : lesquels ?

B2 · La chronologie **11h27 – 11h34 · 7 min**

Objectif : raconter l'histoire du domaine de gauche à droite.

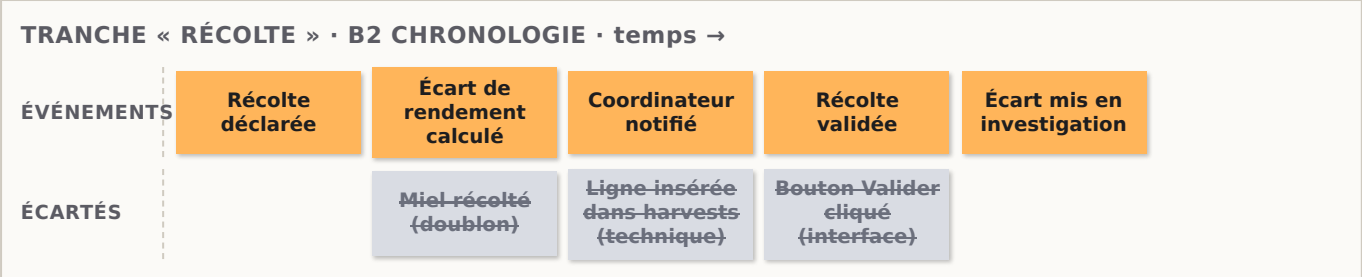
1. Ordonner les événements le long de la flèche du temps. Les événements simultanés ou alternatifs vont l'un au-dessus de l'autre.
2. Fusionner les doublons : garder **le mot du cahier des charges** (« Récolte déclarée » plutôt que « Miel récolté »).
3. Déplacer vers la zone « Écartés » les faits techniques et les détails d'interface (les barrer).
4. Repérer les « événements pivots », qui font changer de phase (par exemple « Alerte levée ») : marquez-les d'une étoile.
5. Quand deux personnes ne sont pas d'accord sur un mot ou sur l'ordre : un post-it **rouge** (point chaud), et on avance.

Une piste pour la flèche du temps (une proposition parmi d'autres)

Avant la saison sites, ruches, comptes	**En continu** mesures, passerelles	**Quand un risque apparaît** détection, alerte	**Sur le terrain** visites, inspections	**À la récolte** déclaration, validation	**En fin de période** indicateurs, rapports

Résultat attendu à 11h34 : une ligne du temps lisible, 40 événements ou plus, des doublons fusionnés, une zone « Écartés » remplie.

Exemple d'interface : la même tranche après la chronologie

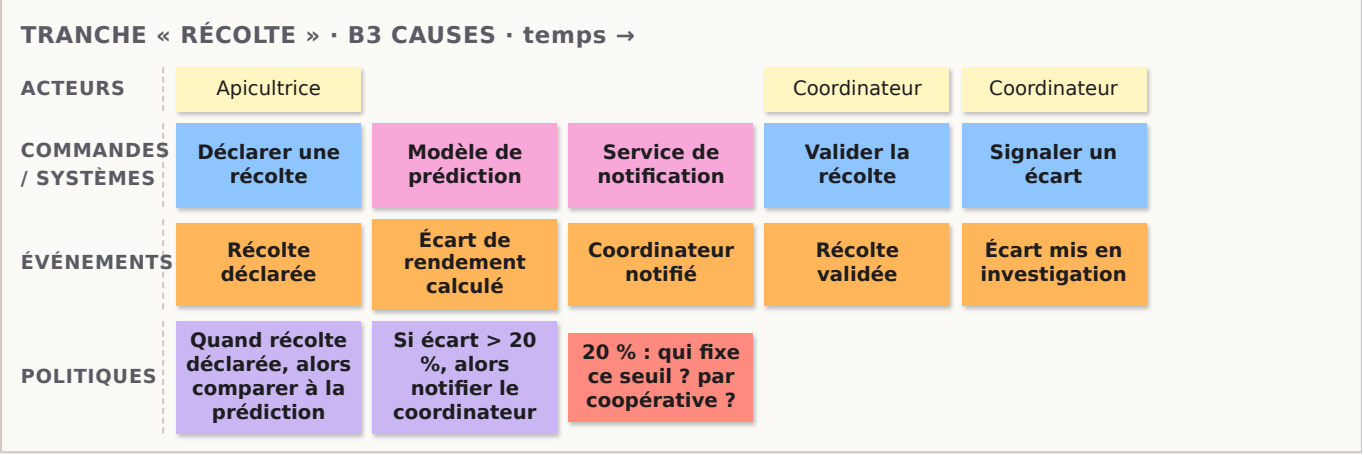


B3 · Les causes 11h34 – 11h42 · 8 min

Objectif : expliquer pourquoi chaque événement se produit.

1. Pour chaque événement, se demander : **qui ou quoi l'a provoqué ?**
   - une personne qui décide : une **commande** bleue à l'impératif (« Déclarer une récolte »), au-dessus de l'événement, avec un petit post-it **acteur** jaune pâle (« Apicultrice ») ;
   - un système extérieur : un post-it **rose** (« Passerelle LoRaWAN », « Modèle de prédiction », « OpenWeatherMap ») ;
   - un autre événement, automatiquement : une **politique** violette sous les deux événements, au format « Quand <événement>, alors <commande> ».
2. Ajouter les politiques avec délai : « Si aucune prise en charge sous 48 h, alors escalader ».
3. Ajouter un point chaud rouge à chaque question sans réponse dans le cahier des charges (seuil, délai, droit, priorité).

Exemple d'interface : la tranche après les causes



Résultat attendu à 11h42 : chaque événement a sa cause ; au moins 4 politiques ; au moins 5 points chauds.

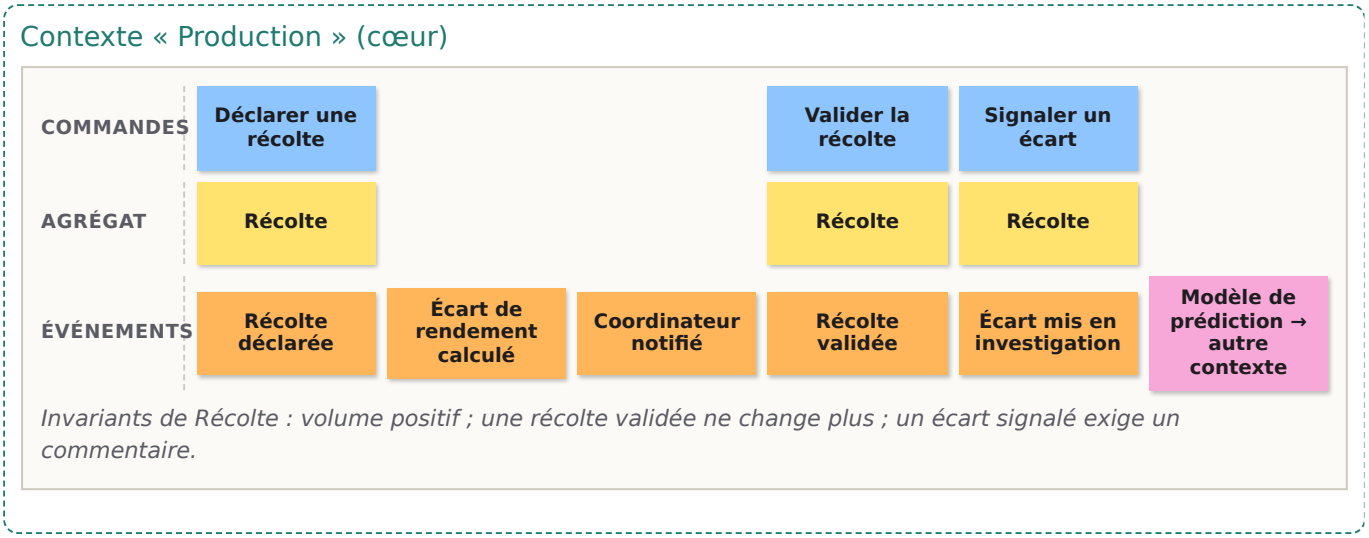
> **Erreur fréquente :** mettre une commande sans acteur, ou un acteur technique (« le serveur »). Un acteur est une personne avec un rôle du cahier des charges (apicultrice, coordinateur régional, gestionnaire, administrateur). Ce qui n'est pas une personne est un système (rose) ou une politique (violet).

B4 · Les agrégats 11h42 – 11h48 · 6 min

Objectif : trouver les objets qui reçoivent les commandes et garantissent les règles.

1. Pour chaque commande, se demander : **quel objet métier doit vérifier les règles avant d'accepter ?** Écrire son nom sur un post-it **jaune**, entre la commande et l'événement.
2. Le même nom revient pour plusieurs commandes ? C'est le même agrégat : regroupez ses commandes et ses événements.
3. Écrire au dos (ou à côté) de chaque agrégat ses **invariants** : « une récolte validée ne peut plus être modifiée », « une alerte résolue n'accepte plus de transition ».
4. Vérifier les règles de taille : un agrégat ne modifie que lui-même ; s'il doit en faire réagir un autre, c'est un événement et une politique.

Exemple d'interface : la tranche avec son agrégat



Résultat attendu à 11h48 : 6 à 12 agrégats, chacun avec 2 invariants écrits. Indice : dans QueenBee, cherchez au moins une alerte, une inspection, une visite, une récolte, une ruche, une passerelle, un rapport.

Partie C · Frontières et glossaire **11h48 – 11h56 · 8 min**

11h15	11h20	11h27	11h34	11h42	11h48	11h56	12h05
A · Préparation	B1 · Chaos	B2 · Chronologie	B3 · Causes	B4 · Agrégats	C · Frontières et glossaire	D · Context map	E · Spécialité

Ce qu'il faut faire

1. Repérer sur le mur les **mots qui changent de sens** : « Alerte », « Ruche », « Site », « Incident », « Validation »… Pour chacun, demander : « est-ce le même cycle de vie, les mêmes personnes, les mêmes règles ? ». Si non, il y a probablement une frontière entre les deux usages.
2. Tracer au feutre (ou avec un cadre Miro en pointillé) **5 à 8 bounded contexts** autour de groupes d'agrégats et d'événements. Chaque post-it appartient à un seul contexte.
3. Nommer chaque contexte avec un nom métier (pas « Service 1 » ni « Backend ») et lui donner un type : **cœur**, **support** ou **générique**.
4. Le scribe complète `docs/glossaire.md` (au moins 15 termes) et `docs/bounded-contexts.md` (modèles en annexe A3).

Exemple de format : glossaire de la tranche « Récolte »

Terme	Définition validée	Contexte	Dans le code	Source
Récolte	Volume de miel déclaré pour une ruche, à une date	Production	`Recolte`	UC-A03
Écart de rendement	Différence, en %, entre la récolte déclarée et la prédiction	Production	`ecartRendement`	UC-A03, UC-C02
Prédiction de rendement	Volume attendu calculé par le modèle pour une ruche et une saison	Prévision	`PredictionRendement`	UC-A03
Validation (d'une récolte)	Acceptation de la déclaration par le coordinateur, après comparaison aux poids mesurés	Production	`recolte.valider()`	UC-C02

Règle : un mot présent dans deux contextes avec deux sens a **deux lignes** dans le glossaire. C'est le signe que votre découpage fonctionne.

Partie D · Context map **11h56 – 12h05 · 9 min**

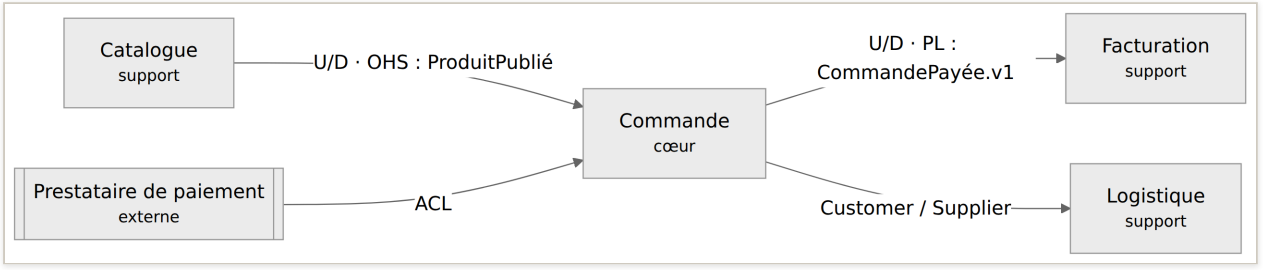
11h15	11h20	11h27	11h34	11h42	11h48	11h56	12h05
A · Préparation	B1 · Chaos	B2 · Chronologie	B3 · Causes	B4 · Agrégats	C · Frontières et glossaire	D · Context map	E · Spécialité

Ce qu'il faut faire

1. Pour chaque événement qui **traverse une frontière** (publié dans un contexte, utile dans un autre), tracer une flèche de l'amont (U, celui qui publie) vers l'aval (D, celui qui consomme).
2. Typer chaque flèche avec une relation du tableau ci-dessous et écrire ce qui est échangé (événement ou API).
3. Repérer les **systèmes externes** (capteurs, passerelles, OpenWeatherMap, messagerie, e-mail) et décider où une couche anticorruption est nécessaire.
4. Écrire `docs/context-map.md` en Mermaid (modèle en annexe A4) et vérifier qu'il s'affiche (aperçu GitHub, GitLab ou mermaid.live).

Relation	Quand l'utiliser	Notation
Open Host / Published Language	L'amont publie une API ou des événements stables et versionnés, pour plusieurs consommateurs	`U/D · OHS`, `PL`
Customer / Supplier	L'aval exprime ses besoins, l'amont les planifie	`C/S`
Conformist	L'aval prend le modèle amont tel quel	`CF`
Anticorruption Layer	L'aval traduit le modèle amont (souvent un système externe) dans le sien	`ACL`
Shared Kernel / Partnership	Petit modèle partagé, ou deux équipes qui livrent ensemble	`SK`, `P`

Exemple d'interface : une context map rendue (sur ShopLite, pour le format)

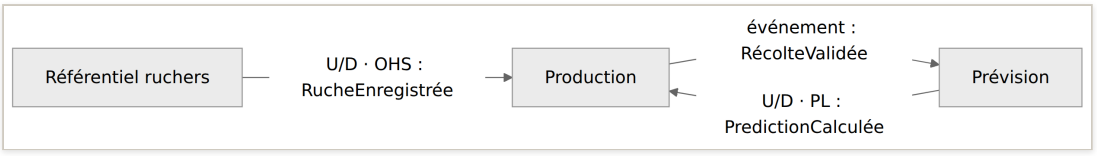


Rendu Mermaid de l'exemple ShopLite (séance 5)

docs/context-map.md (exemple ShopLite)

````
```mermaid
flowchart LR
  CAT["Catalogue<br/><small>support</small>"]
  CMD["Commande<br/><small>cœur</small>"]
  FAC["Facturation<br/><small>support</small>"]
  LOG["Logistique<br/><small>support</small>"]
  PSP[["Prestataire de paiement<br/><small>externe</small>"]]
  CAT -- "U/D · OHS : ProduitPublié" --> CMD
  CMD -- "U/D · PL : CommandePayée.v1" --> FAC
  CMD -- "Customer / Supplier" --> LOG
  PSP -- "ACL" --> CMD
```
````

Exemple d’interface : la tranche « Récolte »



Rendu Mermaid de la tranche « Récolte » vue en cours

Ne recopiez pas ces deux exemples. Votre context map couvre tout QueenBee : 5 à 8 contextes, et au moins 6 relations typées.

Partie E · Suite par spécialité 12h05 – 12h20, puis dossier

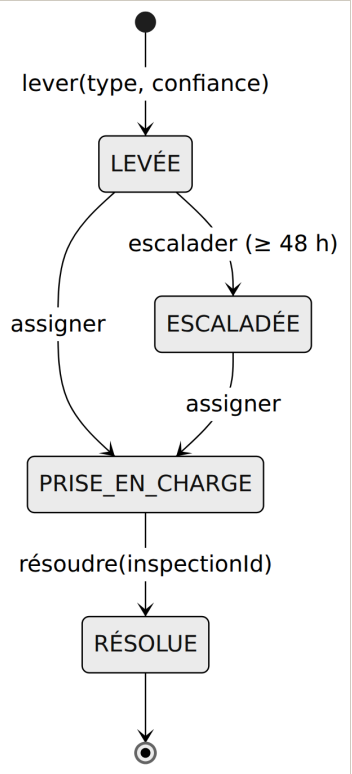
11h15	11h20	11h27	11h34	11h42	11h48	11h56	12h05
A · Préparation	B1 · Chaos	B2 · Chronologie	B3 · Causes	B4 · Agrégats	C · Frontières et glossaire	D · Context map	E · Spécialité

Chaque binôme prolonge le mur dans sa spécialité. En séance, vous **amorcez** le travail (15 min) ; il est terminé pour le dossier du 30/10 et sert de base à votre **ADR de spécialité** (séance 6).

DEV Coder l'agrégat Alerte du contexte « Santé des colonies »

Une alerte est levée par le modèle de prédiction (UC-A02), prise en charge par une apicultrice ou un apiculteur, puis résolue après une inspection (UC-A01). Sans prise en charge sous 48 h, elle est escaladée. Vous appliquez les patterns State et Observer de ce matin, et les règles d'un agrégat.

Fichier du kit `seance5-comportement-ddd/queenbee-alerte/`	**Rôle**
`domain/Alerte.js`	**À compléter** : l'agrégat, ses états (State), ses invariants, ses événements
`application/AlerteService.js`	Fourni : charger → agir → sauvegarder → publier sur l'EventBus
`adapters/InMemoryAlerteRepository.js`	Fourni : un repository en mémoire
`alerte.spec.js` · `demo.js`	Fournis : 12 tests et une démonstration sur deux ruches



Cycle de vie de l'agrégat Alerte (en-tête de domain/Alerte.js)

Au départ : 4 tests sur 12 passent (c’est normal)

```
$ npm run tp5
✔ 1. lever : statut LEVÉE et événement AlerteLevée
✖ 2. invariant : type de risque inconnu refusé
✖ 3. invariant : confiance comprise entre 0 et 1
✔ 4. assigner : PRISE_EN_CHARGE et événement AlertePriseEnCharge
✖ 5. résoudre sans prise en charge : refusé
✖ 6. résoudre après prise en charge : RÉSOLUE
✖ 7. escalader avant 48 h : refusé
✖ 8. escalader après 48 h puis assigner : possible
✖ 9. une alerte résolue n’accepte plus aucune transition
✔ 10. pullEvents vide la liste
✖ 11. service : la politique d’escalade publie AlerteEscaladée
✔ 12. pattern State : aucun switch sur le statut dans l’agrégat
ℹ tests 12
ℹ pass 4
ℹ fail 8
```

Les TODO, dans l’ordre conseillé

TODO	À faire dans `domain/Alerte.js`	Tests
1	Dans `lever()`, refuser un type absent de `TYPES_DE_RISQUE` (message contenant « Type de risque »)	2
2	Refuser une confiance non numérique, ou hors de [0 ; 1] (message contenant « confiance »)	3
3	État `Levee` : `resoudre()` est refusé (message contenant « prise en charge »)	5
4	`estEnRetard(now)` ; `Levee.escalader()` refusé avant 48 h, sinon passage à `Escaladee` et événement `AlerteEscaladée`	7, 8, 11
5a – 5c	État `PriseEnCharge` : refuser `assigner` et `escalader` ; `resoudre(inspectionId)` exige un identifiant, passe à `Resolue`, événement `AlerteRésolue`	6
6a – 6c	État `Escaladee` : `assigner` comme depuis LEVÉE ; refuser `escalader` et `resoudre`	8
7	État `Resolue` : plus aucune transition (trois refus explicites)	9

Contraintes : aucun `switch` sur le statut (le test 12 le vérifie) ; aucun import dans `domain/` (vérifié par `npm run lint:archi`) ; une transition = un changement d’état + un événement, via `alerte._passer(etat, evenement)`, comme l’exemple fourni.

À la fin : 12 tests sur 12, et la règle de dépendance respectée

```
$ npm run tp5
✔ 1. lever : statut LEVÉE et événement AlerteLevée
✔ 2. invariant : type de risque inconnu refusé
  …
✔ 11. service : la politique d'escalade publie AlerteEscaladée
✔ 12. pattern State : aucun switch sur le statut dans l'agrégat
ℹ tests 12
ℹ pass 12
ℹ fail 0

$ npm run lint:archi
✓ Règle de dépendance respectée : aucun fichier domain/ n'importe l'extérieur.
```

La démonstration attendue

```
$ node seance5-comportement-ddd/queenbee-alerte/demo.js
  [2027-03-14T08:00] AlerteLevée         ALR-001 · ruche SN-CASA-014
  [2027-03-14T08:00] AlerteLevée         ALR-002 · ruche CI-TAI-007
  [2027-03-14T14:00] AlertePriseEnCharge ALR-001 · ruche SN-CASA-014
  [2027-03-15T10:00] AlerteRésolue       ALR-001 · ruche SN-CASA-014
  [2027-03-16T16:00] AlerteEscaladée     ALR-002 · ruche CI-TAI-007
  Alertes escaladées : 1
  ✗ refusé : Une alerte doit être prise en charge avant d'être résolue
  [2027-03-16T16:00] AlertePriseEnCharge ALR-002 · ruche CI-TAI-007
```

À rendre (parcours Dev)

☐ `domain/Alerte.js` complété, 12 tests verts, `lint:archi` vert ;

☐ `docs/specialite/dev-agregat-alerte.md` : le diagramme d'états, les invariants, les événements publiés, et **où se trouve cet agrégat sur votre context map** (quel contexte publie `AlerteLevée` ? qui consomme `AlerteEscaladée` ?) ;

☐ pour aller plus loin : un état `ANNULÉE` (fausse alerte signalée par le coordinateur), avec ses tests ; un observateur qui « notifie » le coordinateur du site en cas d'escalade.

> **Vers l'ADR de spécialité :** « Comment l'application terrain hors ligne reçoit-elle les alertes ? » (push, synchronisation au prochain accès, file locale) ou « Où vit la politique d'escalade à 48 h ? » (tâche planifiée, file différée, base).

DATA Le contrat de données de l'événement « Mesure validée »

La mesure produite par la collecte (UC-T02) traverse une frontière : elle alimente la santé des colonies, la prédiction et l'impact. Vous écrivez le **contrat** de cet événement, c'est-à-dire son « published language » sur la context map.

Ce qu'il faut faire

1. Identifier sur votre context map le contexte **propriétaire** de l'événement et ses **consommateurs**.
2. Compléter le tableau des champs à partir du format 6.5 du cahier des charges : type, unité, plage physique valide, obligatoire, règle de validation.
3. Écrire l'enveloppe de l'événement (identifiant, type, version, date, producteur) et un exemple JSON valide.
4. Fixer les règles : clé de déduplication, traitement d'une trame hors plage ou rejouée (UC-CY02), fréquence (1 mesure / 15 min, 1 / 5 min en surveillance renforcée), retard maximal accepté

(mode dégradé, UC-CY01), versionnement.

Exemple d'interface : le début du tableau (à compléter)

Champ	Type	Unité	Plage valide	Oblig.	Règle / remarque
`hive_id`	string	–	format `PP-SITE-NNN`	oui	ruche connue du référentiel, sinon rejet
`timestamp`	string ISO 8601 UTC	–	≤ maintenant + 5 min	oui	avec `hive_id` : clé de déduplication
`weight_kg`	number	kg	à définir (ex. 0 – 120)	oui	variation brutale : signal de risque ou d'erreur ?
`temp_internal_c`	…	…	…	…	…
`position_lat` · `position_lon`	…	…	…	…	lien avec UC-CY04 (déplacement)

Exemple d'interface : l'enveloppe de l'événement (à adapter)

`docs/specialite/data-contrat-mesure-validee.md · exemple JSON`

```
{
  "eventId": "6f1c2a9e-5b7d-4c1e-9a3f-2d8e0b4c7a11",
  "type": "MesureValidée",
  "version": 1,
  "occurredAt": "2027-03-14T08:15:00Z",
  "receivedAt": "2027-03-14T08:41:12Z",
  "producer": "<contexte propriétaire>",
  "payload": {
    "hive_id": "SN-CASA-014",
    "site_id": "SN-CASA",
    "timestamp": "2027-03-14T08:15:00Z",
    "weight_kg": 34.2,
    "temp_internal_c": 34.8,
    "temp_external_c": 27.1,
    "humidity_pct": 58,
    "acoustic_activity_score": 0.72,
    "battery_pct": 91,
    "position_lat": 12.5833,
    "position_lon": -16.2833
  }
}
```

Pourquoi deux dates ? `occurredAt` est la date de la mesure ; `receivedAt` la date de réception au central, parfois des heures plus tard en mode dégradé. Quel est l'impact sur les alertes ?

À rendre (parcours Data)

☐ `docs/specialite/data-contrat-mesure-validee.md` : propriétaire, consommateurs, tableau complet des champs, enveloppe, exemple JSON, règles de déduplication, de rejet et de versionnement ;

☐ la flèche correspondante sur la context map, avec `PL : MesureValidée.v1` ;

☐ l'amorce de l'ADR : « Ingestion des mesures : traitement par lots ou en flux ? » (contexte, options, critères : latence des alertes, connectivité intermittente, volumétrie de 38 400 mesures par jour).

CYBER Les zones de confiance sur la context map

Chaque frontière de contexte est aussi une frontière de confiance potentielle. Vous annotez votre context map avec les zones réseau du cahier des charges (section 6.3 : DMZ, LAN applicatif, IoT, Management) et les mécanismes d'authentification à chaque passage.

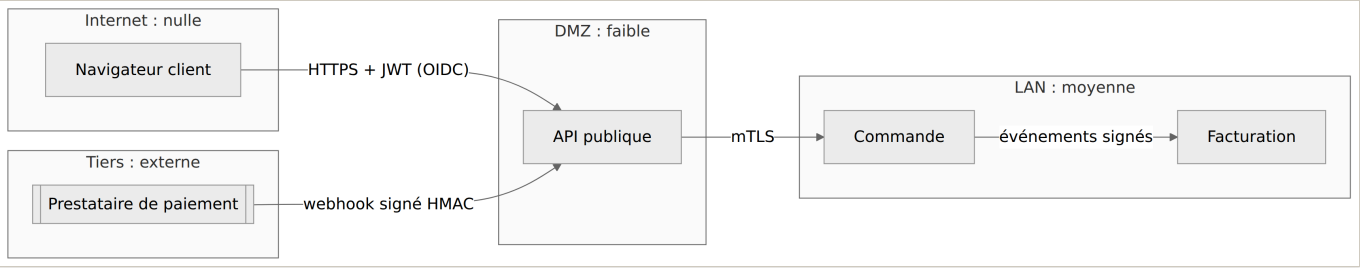
Ce qu'il faut faire

1. Lister les **points d'entrée de la donnée** : capteurs, passerelles de site, application terrain hors ligne, interfaces d'administration, API du gestionnaire.
2. Placer chaque bounded context dans une zone et lui donner un niveau de confiance (faible, moyen, élevé).
3. Pour chaque flèche qui change de zone : qui s'authentifie, comment (clés LoRaWAN, certificats mTLS, JWT et MFA, signature des événements), et ce qui est journalisé.
4. Relier les incidents UC-CY01 à UC-CY04 aux contextes concernés et aux événements de votre mur (passerelle muette, trame rejouée, brouillage, ruche déplacée).

Exemple d'interface : le tableau des zones (début)

Contexte	Zone	Confiance	Entrées de données	Authentification à l'entrée	Menace principale
Supervision et sécurité	Management (VLAN 30)	élevée	journaux des passerelles	mTLS + comptes nominatifs + MFA	compromission d'un compte admin
…	…	…	…	…	…

Exemple d'interface : une context map annotée de zones (sur ShopLite, pour le format)



Rendu Mermaid : sous-graphes = zones, étiquettes = authentification au passage

docs/specialite/cyber-zones-de-confiance.md · exemple ShopLite

````
```mermaid
flowchart LR
  subgraph INTERNET["Internet : nulle"]
    CLIENT["Navigateur client"]
  end
  subgraph DMZ["DMZ : faible"]
    API["API publique"]
  end
  subgraph LAN["LAN : moyenne"]
    CMD["Commande"]
    FAC["Facturation"]
  end
  subgraph TIERS["Tiers : externe"]
    PSP[["Prestataire de paiement"]]
  end
  CLIENT -- "HTTPS + JWT (OIDC)" --> API
  API -- "mTLS" --> CMD
  CMD -- "événements signés" --> FAC
  PSP -- "webhook signé HMAC" --> API
```
````

À rendre (parcours Cyber)

☐ `docs/specialite/cyber-zones-de-confiance.md` : le tableau complet des contextes et des zones, la context map annotée en Mermaid (sous-graphes = zones), la liste des passages de frontière avec leur authentification ;

☐ les incidents UC-CY01 à UC-CY04 reliés aux contextes et aux événements du mur ;

☐ l'amorce de l'ADR de sécurité : par exemple « Authentification des passerelles de site auprès du central : clé partagée, certificats mTLS ou VPN seul ? » et la liste des menaces à analyser (STRIDE) en priorité.

7 · Rendu, auto-vérification et grille

L’arborescence à déposer (par binôme)

```
queenbee-archi-<nom1>-<nom2>/
├── README.md                      ← spécialité, membres du groupe, état d’avancement
├── docs/
│   ├── event-storming/
│   │   ├── mur-complet.jpg        ← photo nette (ou export PDF du tableau Miro)
│   │   └── mur-<contexte>.jpg     ← zooms lisibles, un par contexte si besoin
│   ├── glossaire.md
│   ├── bounded-contexts.md
│   ├── context-map.md
│   ├── points-chauds.md
│   └── specialite/
│       ├── dev-agregat-alerte.md            (Dev, + le code du kit complété)
│       ├── data-contrat-mesure-validee.md   (Data)
│       └── cyber-zones-de-confiance.md      (Cyber)
└── adr/                           ← séance 6 : 3 à 5 ADR, dont un de spécialité
```

Avant 12h20 : la liste de vérification

☐ Tous les événements sont des faits métier au passé, compréhensibles par une apicultrice ou un gestionnaire.

☐ Chaque événement a une cause : commande et acteur, système externe ou politique.

☐ Chaque agrégat a un nom métier et au moins deux invariants écrits.

☐ Chaque bounded context a un nom métier, un type (cœur, support, générique) et des UC sources.

☐ Le glossaire compte au moins 15 termes ; « Alerte » y est tranché (une ou deux lignes ? pourquoi ?).

☐ La context map s’affiche, et chaque flèche porte un type de relation et ce qui est échangé.

☐ Les points chauds sont listés avec la personne qui pourrait y répondre (client, BeeAfrica, équipe).

☐ Le livrable de spécialité est amorcé et sa suite est notée dans le README.

Grille d’évaluation du TP 5 (intégrée au dossier du 30/10)

Critère	Points
Mur : événements métier au passé, chronologie, causes complètes, couverture des UC	5
Glossaire : définitions précises, mots ambigus tranchés, noms dans le code	3
Bounded contexts : frontières justifiées par le vocabulaire, types de sous-domaines	4
Context map : relations typées, échanges nommés, systèmes externes et ACL	4
Livrable de spécialité (Dev, Data ou Cyber), terminé pour le dossier	4
Total	**20**

Annexe A1 · Les cas d’utilisation en bref

Résumé du chapitre 4 du cahier des charges QueenBee. En cas de doute, le cahier des charges fait foi.

Cas	Acteur	Ce qui se passe
UC-A01	Apicultrice	Enregistre une inspection, même hors ligne (état visuel, reine, comportement, photo). Synchronisation au retour du réseau ; le coordinateur est notifié en cas d’anomalie. Exceptions : conflit de synchronisation, ruche introuvable.
UC-A02	Apicultrice	Le système détecte un risque (chute de poids, activité acoustique atypique) ; alerte envoyée (push, ou au prochain accès) avec type de risque, niveau de confiance et recommandation ; visite anticipée possible.
UC-A03	Apicultrice	Déclare une récolte (ruche, volume, date) ; comparaison à la prédiction de rendement ; coopérative et coordinateur notifiés.
UC-C01	Coordinateur	Consulte les ruches classées par risque, planifie les visites selon la criticité et la géolocalisation, les assigne.
UC-C02	Coordinateur	Consulte les récoltes en attente, compare aux poids mesurés, valide ou signale un écart à investiguer.
UC-G01	Gestionnaire	Consulte les indicateurs multi-sites (rendement, colonies à risque, couverture réseau), compare sites et pays.
UC-G02	Gestionnaire	Choisit période, sites et indicateurs ; rapport d’impact PDF généré, téléchargé ou envoyé au bailleur.
UC-AD01	Administrateur	Crée, modifie, désactive des comptes ; assigne rôles et permissions par site ; toutes les actions sont tracées.
UC-T01	Tous	Authentification (MFA pour coordinateurs, gestionnaires, admins), jeton adapté au hors-ligne, données locales chiffrées ; blocage après 5 échecs.
UC-T02	Capteur	Mesures (poids, températures, humidité, acoustique) toutes les 30 min ou immédiatement si seuil dépassé ; transmission LoRaWAN vers la passerelle, puis en différé vers le central.
UC-CY01	Supervision	Passerelle sans battement de cœur depuis 15 min : alerte de site, mode dégradé (cache local), resynchronisation au rétablissement.
UC-CY02	Ingestion	Trame hors plage physique ou déjà reçue (rejeu) : rejetée et journalisée ; alerte d’intégrité si répétée.
UC-CY03	Passerelle	Taux anormal de trames perdues sur un canal : alerte de brouillage, changement de canal proposé, incident journalisé.
UC-CY04	Ingestion	Position GPS hors tolérance : déplacement déclaré (transhumance) ou non ; si non déclaré, alerte prioritaire au coordinateur et à l’administrateur.

Annexe A2 · Les événements de départ

Seize amorces pour ne pas partir d’une page blanche, plus quatre post-it à placer. Elles ne suffisent pas : complétez, réordonnez, regroupez.

ÉVÉNEMENTS DE DÉPART

COLLECTE (UC-T02, UC-CY02)	**Mesure reçue**	**Trame rejetée**	**Capteur devenu silencieux**	**Passerelle hors ligne détectée**
SANTÉ DE LA COLONIE (UC-A02)	**Seuil dépassé**	**Risque d’essaimage détecté**	**Score de santé recalculé**	**Alerte levée**
TERRAIN (UC-A01, UC-C01)	**Visite planifiée**	**Inspection enregistrée hors ligne**	**Inspection synchronisée**	**Conflit de synchronisation détecté**
IMPACT (UC-G01, UC-G02)	**Indicateurs consolidés**	**Période de rapport choisie**	**Rapport d’impact généré**	**Rapport envoyé au bailleur**
À PLACER	**Capteurs + passerelle LoRaWAN**	**OpenWeatherMap**	**Déplacement non déclaré détecté**	**« Alerte » : colonie ou infrastructure ? Même mot, deux sens ?**

Annexe A3 · Modèles de fichiers

docs/glossaire.md

```
# Glossaire QueenBee · groupe <spécialité>-<n>

| Terme | Définition validée | Contexte | Dans le code | Source |
|---|---|---|---|---|
| Récolte | Volume de miel déclaré pour une ruche, à une date | Production | `Recolte` | UC-A03 |
| … | … | … | … | … |

## Mots à plusieurs sens
- « … » : sens 1 dans <contexte A> ; sens 2 dans <contexte B>. Décision : …
```

docs/bounded-contexts.md

```
# Bounded contexts QueenBee

## <Nom du contexte>
- Responsabilité (une phrase) : …
- Type : cœur | support | générique · spécialité la plus concernée : Dev | Data | Cyber
- Agrégats : … (invariants principaux : …)
- Événements publiés : …
- Événements consommés : … (venant de …)
- Cas d'utilisation sources : UC-…
- Mots propres à ce contexte : …
```

docs/points-chauds.md

```
# Points chauds

| # | Question | Impact si on se trompe | Qui peut répondre | Devient un ADR ? |
|---|---|---|---|---|
| 1 | Seuil d'écart de rendement : 20 % ? par coopérative ? | Alertes inutiles ou manquées | Coordinateurs BeeAfrica | non (paramètre) |
| 2 | … | … | … | … |
```

Annexe A4 · Aide-mémoire Mermaid pour la context map

docs/context-map.md · squelette

````
# Context map QueenBee · groupe <spécialité>-<n>

```mermaid
flowchart LR
  A["Contexte A<br/><small>cœur</small>"]
  B["Contexte B<br/><small>support</small>"]
  X[["Système externe"]]
  A -- "U/D · PL : ÉvénementPublié.v1" --> B
  X -- "ACL" --> A
```

## Relations
| Amont | Aval | Relation | Échange | Justification |
|---|---|---|---|---|
````

Syntaxe	Effet
`A["Nom affiché"]`	Un contexte (rectangle)
`S[["Système externe"]]`	Un système externe (rectangle à double bord)
`A -- "texte" --> B`	Une flèche étiquetée de A (amont) vers B (aval)
`A -. "texte" .-> B`	Une flèche en pointillé (relation faible ou future)
`subgraph Z["Zone"] … end`	Un regroupement (zone de confiance, équipe)
` `, `<small>…</small>`	Retour à la ligne, petit texte dans un libellé

Vérifier le rendu : GitHub et GitLab affichent les blocs ```` ```mermaid ```` d'un fichier Markdown. Sinon, collez le code dans l'éditeur en ligne mermaid.live et exportez une image pour le dossier.

Annexe A5 · Modèle d'ADR (pour la séance 6)

adr/0001-titre-court.md

```
# ADR 0001 · <décision, en une phrase>
- Statut : proposé | accepté | remplacé par ADR-…
- Date : 2026-10-23 · Auteurs : …

## Contexte
<le problème, les contraintes du cahier des charges, le point chaud d'origine>

## Options envisagées
1. … (avantages, inconvénients)
2. …

## Décision
<l'option retenue et pourquoi>

## Conséquences
<ce qui devient plus facile, plus difficile ; ce qu'il faudra surveiller>
```